Benchmarking Canonical Matrix Multiplication Across Paradigms

Evaluating AOT (C), JIT (Java HotSpot), and Interpreted (CPython) Runtimes Under Identical Constraints

Research Objective & Theoretical Model

Core Question: How does matrix dimension n govern runtime and memory when algorithmic structure ($i-j-k$), precision (IEEE 754 float64), and input data are held strictly identical?

Kernel: Canonical triple-loop: $C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj} \rightarrow 2n^3$ FLOPs
Inputs: Bit-for-bit identical 64-bit SplitMix64 PRNG ($S = 42$) across all languages
Timing: Isolated monotonic clock around **only** the inner arithmetic loop

Hardware Specifications & Controlled Single-Threaded Testbed

HOST PROCESSOR

Apple M2 (ARMv8.5-A)

4P (3.5GHz) + 4E (2.4GHz)

16 MB Shared L2 | Single-threaded

MEMORY & SOFTWARE

8.0 GB Unified Memory

100 GB/s LPDDR5-6400

Clang 21.0 | JDK 22.0.2 | Python 3.14

Experimental Benchmarking Protocol

n = 32 - 2048
Matrix Dimensions
10 discrete matrix sizes

R = 5 runs
Median + IQR
Robust to system noise

W = 1 - 3
Dedicated Warm-ups
Reaching JIT steady-state

60.0 s
Timeout Budget
Per repetition cutoff

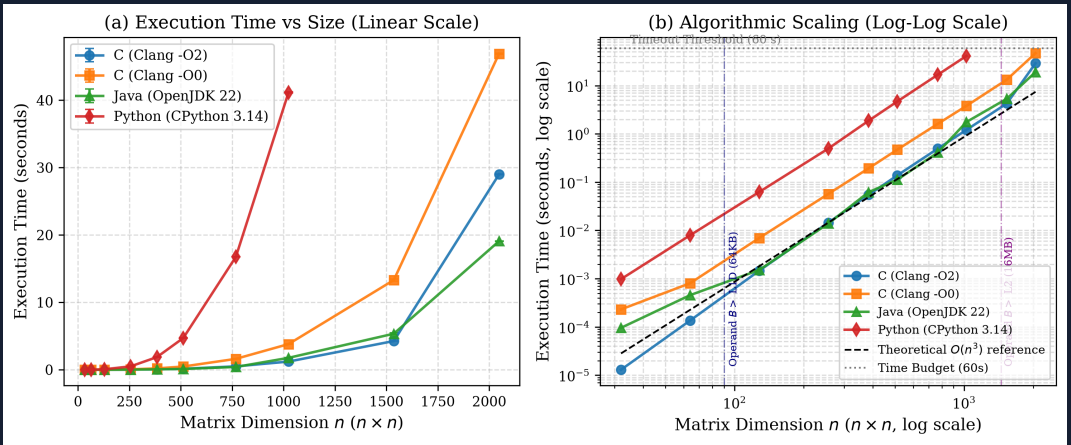
Integrity & Defensive Design:

- **Dead-Code Elimination Defense:** Matrix checksum $S = \sum C_{ij}$ verified after timing.
- **Zero Allocation in Loop:** Flat buffers allocated prior to timing capture.
- **Ground Truth:** Relative error $< 10^{-12}$ validated against NumPy BLAS ($A \times 0$, $A \times I$).

Execution Time Divergence & Feasibility Wall

Cubic Complexity Holds ($O(n^3)$), but Constant Factors Diverge by Over Two Orders of Magnitude

Empirical Runtime: Linear Scale (a) & Log-Log Scaling (b)



Execution Timings at $n = 1024$

1.21 s
C (Clang -O2)
Baseline (1.78 GFLOPS)

1.76 s
Java (JDK 22)
1.45x slowdown vs C

3.80 s
C (Clang -O0)
3.15x slowdown vs C -O2

41.15 s
Python 3.14
34.0x slower than C

Empirical Scaling Exponents ($T(n) = c \cdot n^{\alpha}$)

- **C (-O2):** $\alpha = 3.38$ ($R^2 = 0.995$) – DRAM stride penalty
- **C (-O0):** $\alpha = 3.10$ ($R^2 = 0.999$)
- **Java:** $\alpha = 3.13$ ($R^2 = 0.990$)
- **Python:** $\alpha = 3.10$ ($R^2 = 1.000$)

Constant Factor Overhead ($\frac{c_{\text{Python}}}{c_{\text{C-O2}}}$):

$1.91 \times \frac{10^{-8}}{1.01} \times 10^{-10} = \mathbf{189}\times$ **baseline gap**

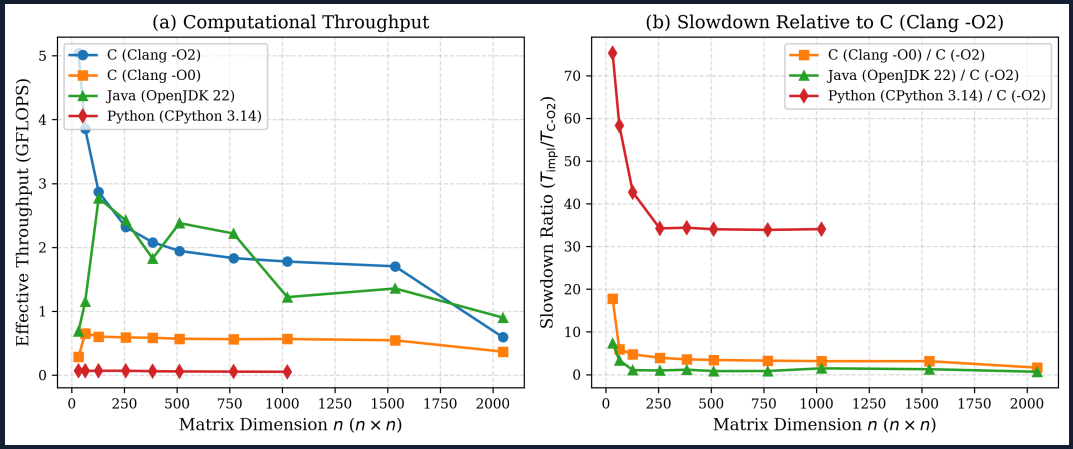
CRITICAL DISCOVERY: THE 60-SECOND TIMEOUT WALL

At $n = 1024$, Python takes **41.15s**. At $n = 1536$, cubic projection predicts **139s**, exceeding the budget and triggering process abortion. **Python's operational ceiling is strictly $n = 1024$**, whereas C and Java scale seamlessly to $n = 2048$.

Throughput Analysis & The Power of JIT Compilation

Understanding Why Python Stalls and How Java HotSpot Matches Native Compiled C

Computational Throughput (GFLOPS) & Relative Slowdown



Python 0.06 GFLOPS Bottleneck

- **Interpreter Dispatch:** Inner loop opcode decoding in `ceval.c`.
- **Dynamic Object Boxing:** Every number is a heap `PyFloatObject` (24 bytes).
- **Pointer Indirection:** Stalls hardware CPU execution pipeline.

Java HotSpot C2 JIT Parity

- **Tier 4 Compilation:** Unrolls inner loops and allocates registers.
- **ARM64 NEON FMA:** Generates fused multiply-accumulate vector instructions.
- **Outperforms C at $n = 2048$:** 19.1s (Java) vs 29.0s (C) due to loop structure.

Optimization Impact: C -O2 vs -O0

3.15x

Compiler Speedup

-O2 vs -O0 at n = 1024

Why is -O0 so slow?

- Accumulator `sum` is spilled to RAM stack frame on **every iteration** of k.
- No instruction scheduling or loop unrolling.

Crucial Takeaway:

Even unoptimized C (`-O0`) is **10.8x faster than Python**, proving that native compilation without dynamic boxing provides an immense baseline advantage.

Warm-Up Impact on Java

3.5x

Warm-Up Speedup

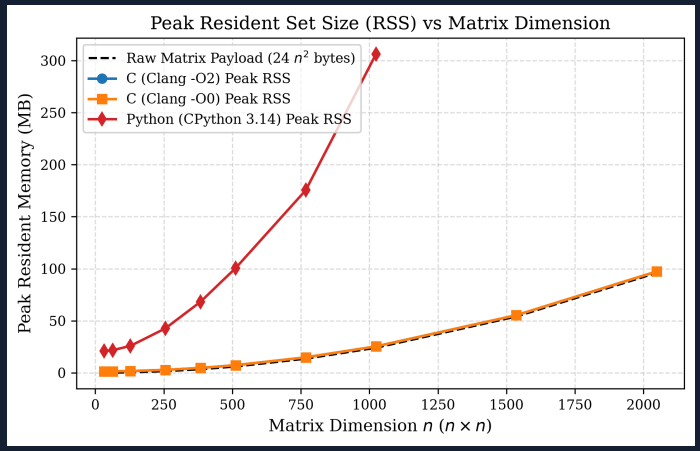
Cold run vs Steady-state C2

Without warm-up, Java's first run is slowed down by Tier 0 bytecode interpretation and C1 profiling.

Memory Overhead & Practical Takeaways

Data Representation Dictates Memory Expansion and Hardware Cache Stride Bottlenecks

Memory Scaling: Process Peak RSS vs Raw Payload ($24n^2$ bytes)



Memory Footprint at $n = 1024$

24.0 MB **Raw Payload** 3 matrices x 8 MB ($24n^2$)	**25.3 MB** **C Peak RSS** 1.05x raw payload
23.8 MB **Java Heap** Managed delta (†)	**306.3 MB** **Python RSS** 12.8x raw payload expansion

† Java RSS includes JVM metaspace and runtime (80MB base).

THE CACHE STRIDE WALL (THE $i - j - k$ BOTTLENECK)

In row-major layout, accessing $B[k][j]$ along columns incurs a stride of $8n$ bytes. Once $8n > 64$B (cache line), **every iteration triggers a cache miss**. At $n = 2048$, Matrix B consumes 32 MB, overflowing the 16 MB L2 cache and triggering steep DRAM latency stalls.

Core Takeaways for Data Engineering

1. **Never use pure Python loops** for numerical linear algebra; always rely on compiled C/C++ kernels (NumPy BLAS) or JIT runtimes.
2. **Java HotSpot is production-grade** for high-throughput numeric engines when warm-up and flat arrays are leveraged.
3. **Hardware awareness is mandatory:** Algorithmic $O(n^3)$ math cannot mask CPU cache hierarchy physics.
4. **Bridge to Assignment 2:** Cache-tiling and loop reordering ($i - k - j$) will break through this cache stride wall.